UNIVERSIDAD AMERICANA
FACULTAD DE INGENIERÍA Y ARQUITECTURA

---




**Ingeniería en Sistemas de Información**

---

# Servicios Web

---

**Estudiante:**

**Emilio Jose Carranza Rivera**

**Profesor:**

**José Duran Garcia**

**Preguntas de Reflexión y Respuestas**

1. **¿Por qué el arreglo compartido pertenece al proveedor y no a cada tarjeta?**

   El arreglo compartido pertenece a `Solicitudes Provider` para mantener una **única fuente de verdad** (*Single Source of Truth*). Si el estado residiera en componentes individuales o tarjetas, sería imposible sincronizar datos globales entre componentes hermanos, como el contador de solicitudes pendientes en el panel superior y la lista filtrada de elementos en la interfaz.

2. ¿Qué cambia entre `useState` y `useReducer` en este caso?

   useState` es ideal para gestionar estados locales sencillos e independientes (como el texto de un input o el estado del filtro activo). Por el contrario, `useReducer` es preferible cuando la lógica de estado es compleja e involucra múltiples transiciones relacionadas (`CARGANDO`, `CARGADAS`, `ERROR`, `AGREGADA`, `ACTUALIZADA`, `ELIMINADA`), permitiendo centralizar y controlar todas las reglas de actualización dentro de una función pura de manera predecible.

3. ¿Para qué se utiliza `AbortController` al cargar solicitudes?

   `AbortController` se utiliza para **cancelar peticiones HTTP asíncronas en curso** mediante su propiedad `signal`. Si el componente se desmonta mientras Axios sigue esperando la respuesta del servidor, el método de limpieza de `useEffect` llama a `controlador.abort()`, evitando fugas de memoria (*memory leaks*) e intentos de actualizar el estado en componentes no montados.

4. ¿Por qué no es correcto considerar que `useEffect` equivale exactamente a todos los métodos de ciclo de vida de un componente de clase?

   useEffect` no es un reemplazo directo de `componentDidMount`, `componentDidUpdate` o `componentWillUnmount`. Su filosofía se basa en la **sincronización con sistemas externos** (efectos secundarios) en función del cambio de dependencias declaradas, más que en reaccionar a etapas fijas del ciclo de vida del componente.

5. ¿Qué limitación tiene guardar los datos en una lista en memoria?

   La principal limitación es la **falta de persistencia de datos**. Toda la información creada, modificada o eliminada se pierde al reiniciar o detener la aplicación de Spring Boot, volviendo siempre al estado inicial declarado en el código. Para producción se requeriría integrar un sistema de gestión de base de datos relacional (como PostgreSQL o MySQL) a través de Spring Data JPA.